

Heuristic Weighted Bandwidth Algorithms: The State of the Research

Alexander Matheson¹

August 2026

1 Introduction

The bandwidth of some unweighted graph G with n nodes and m edges is the maximum integer $k \in \{0, 1, \dots, n-1\}$ such that $|i-j| > k$ for all adjacent $i, j \in \{0, 1, \dots, n-1\}$. The bandwidth of a weighted graph with n nodes and m edges is the maximum of $w_{i,j}|i-j|$ for all adjacent $i, j \in \{0, 1, \dots, n-1\}$ where w is the weight of the edge from i to j . More specifically: let π be a bijection from the set of nodes in G to $\{0, 1, \dots, n-1\}$, E be the list of all edges in G , and $w_{i,j}$ be the weight of the edge from i to j . The weighted bandwidth of G under π is given by $\beta(\pi) = \max_{i,j \in E} (w_{i,j}|\pi(i) - \pi(j)|)$. Note that graph bandwidth is equivalent to matrix bandwidth. Finding the minimum bandwidth across all bijections is NP-complete and is thus inefficient for large graphs [Pap76]. Therefore, various heuristic algorithms have been developed. These algorithms do not guarantee perfect accuracy, but provide response that is "good enough" for most cases. However, the vast majority of these algorithms do not work for, or generalize naïvely to, weighted graphs [Var26]. There exists also the possibility of "metaheuristic" algorithms, which would use iterative search frameworks to find solutions (i.e. treat some π as a candidate solution and evaluate it with a given objective) and thus generalize less naïvely to weighted bandwidth, but are (in theory) slower than regular heuristics [Var25] [MS14]. These could provide a baseline with which to compare the thesis' main algorithm.

2 Application

Bandwidth reduction is particularly useful in circuit design and has various other applications in VLSI layout [LLX06, MS14, Var25, Var26]. A circuit can be represented as a graph in which vertices correspond to circuit cells and edges correspond to connections between cells. However using unweighted bandwidth would imply that each of the connections between cells are equally important for the circuit design, which does not reflect reality in most cases; some connections are substantially more "timing-critical" than others. Weighted edges can more accurately model timing-criticality [Var26].

3 Classical Bandwidth Heuristics

Of the existing bandwidth heuristics, Cuthill–McKee is one of the more foundational. The Cuthill–McKee algorithm works by first using a heuristic to choose some "pseudoperipheral" node [GL79], that is a node

¹Department of Mathematics & Computer Science, Mount Allison University, Sackville, NB, Canada E4L 1E4

that is close to peripheral, as a starting node and adds this node to a list L , the algorithm then iterates through the following steps for $i \in \{0, 1, \dots, n - 1\}$:

- List all adjacent nodes of L_i not already in L .
- Sort these nodes by ascending degree.
- Append this list to L .
- Increment i .

The nodes are then labeled by their index in L [CM69].

After these steps are completed, we are left with a level structure, where all the nodes in level j have a minimum distance from the starting node of j . The nodes in each level are ordered by their predecessor's index in L , and nodes with the same predecessor are ordered by their degree [CM69]. George notes that reversing this order often produces a better profile, but does nothing for the bandwidth of a graph [Geo71], the profile being the sum of the distances from each diagonal entry to the leftmost nonzero entry in that row.

This heuristic can be naïvely extended to weighted bandwidth by using weighted distance to build the level structure, however this extension ignores high weight edges between nodes within the same level, which contribute greatly to bandwidth. Below is an example of this issue.

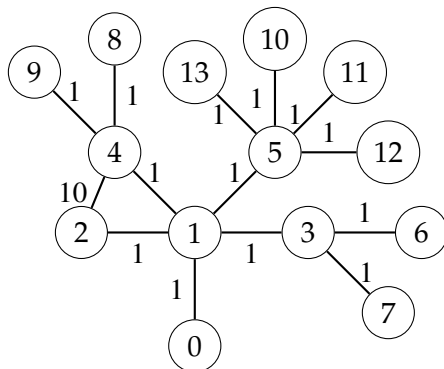


Figure 1: Suboptimal layout given by weighted Cuthill–McKee, $\beta = 20$.

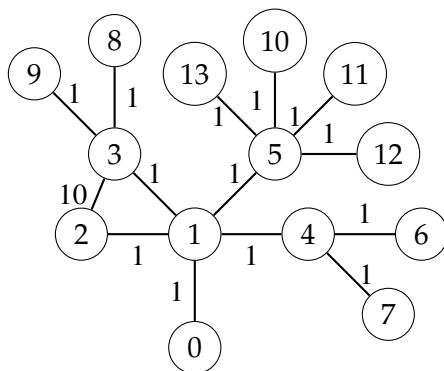


Figure 2: An optimal layout of the same graph, $\beta = 10$.

The “GPS” heuristic presented in [GPS76] is comparable to (reverse) Cuthill–McKee. The Cuthill–McKee

heuristic depends heavily on the choice of the starting node, GPS attempts to circumvent this by essentially choosing two starting peripheral nodes distant from each other, generating a level structure from each node, then combining the two level structures [GPS76]. This heuristic still suffers from the same challenge as Cuthill–McKee when applied to weighted graphs using weighted distance to generate the level structure, but this problem could possibly be mitigated by choosing more starting nodes. As a note, it is also possible that weighted structure could somehow produce *worse* diminishing returns. This would require testing and probably a weight-aware heuristic to choose starting nodes.

4 Existing Weighted Bandwidth Literature

To my knowledge, there are a total of three papers on weighted bandwidth. [LLX06] presents a characterization but no explicit algorithm. [SRB08] presents an approach that can extend to the weighted bandwidth problem but is not weight-aware. [NVDSCD24] does present a weight-aware algorithm for bandwidth recognition, however it is exact and, as mentioned above, slow and infeasible for large graphs.

5 Varona’s Proposed Method

An improvement to this naïve generalization is discussed by Varona at ECCC 2026 [Var26] wherein a linear forest is first constructed by selecting some threshold t , then ordering all edges with weight greater than t by descending weight. We construct the paths in this linear forest as follows:

- Include the greatest weight edge not in the linear forest.
- For both nodes at the ends of the path, add the greatest weight edge with weight greater than t to the path, if any exist.
- Repeat these steps until the path stops growing.

The paths in the linear forest are then each contracted into a single node. Reverse Cuthill–McKee is then run with the weighted generalization discussed above, uses the custom solver developed by Varona and presented at ECCC 2026 [Var26]. The contracted nodes are then expanded back into their original paths [Var26].

This, in practice, performs better on “heavy backbone” graphs (e.g., caterpillars and in fact VLSI circuits). However there are still questions to answer and possible improvements to be made [Var26].

6 Scope of the Thesis

Here are some things to work on:

- How do we compute t ?
- Is there a better linear forest construction we can use?
- When expanding the contracted nodes, labels are assigned sequentially from one of the endpoints. Which endpoint do we choose, how do we decide this computationally? At the moment coordinate gradient descent looks promising.

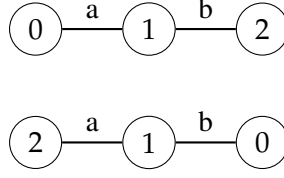


Figure 3: The two possible expansions of three contracted nodes assigned index 0.

- When a path within the linear forest is contracted, the weights of edges from nodes outside of this path to nodes within the path need to be recomputed to properly represent the "location" of their endpoint in the path. Essentially we need to somehow include the edge's distance from the path's endpoints in the weighted heuristic. There are two cases to consider here, one where multiple nodes along the path have an edge to one node outside the path, and the other when a node not at the endpoint of the path has an edge to a node outside the path. Varona believes that somehow multiplicatively including the weights of the edges between the path's endpoints and the outside edge's endpoint could work.

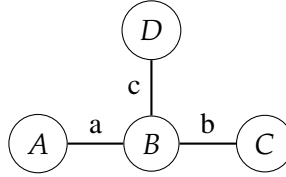


Figure 4: The weighted Cuthill-McKee heuristic, when applied after nodes A , B , and C are contracted would assume outside node D is adjacent to an endpoint of the path if edge weights are not recomputed.

- Is there a weight aware method of selecting the starting node? Something perhaps aware of the graph's topology and using more than just weighted distance?
- Could using multiple starting nodes produce better results? Similar to what was discussed in [GPS76] for unweighted graphs. How would this interact with the multilevel approach mentioned below?
- In [GPS76], it was noted that using multiple starting nodes produced diminishing returns, however it is possible that using multiple starting nodes for weighted graphs will produce less diminishing returns.
- Would selecting multiple threshold levels and running the contraction step for each threshold level improve upon this heuristic? A similar method is discussed in [SRB08].
- While not necessarily within the scope of this thesis, benchmarking on VLSI circuits could be the subject of a future paper.

References

- [CM69] Elizabeth Cuthill and James McKee. Reducing the bandwidth of sparse symmetric matrices. In *Proceedings of the 24th National Conference of the ACM*, pages 157–172. Brandon Systems Press, 1969. 2

- [Geo71] J. Alan George. *Computer Implementation of the Finite Element Method*. PhD thesis, Department of Computer Science, Stanford University, 1971. [2](#)
- [GL79] J. Alan George and Joseph W. H. Liu. An implementation of a pseudoperipheral node finder. *ACM Transactions on Mathematical Software*, 5(3):284–295, 1979. [1](#)
- [GPS76] Norman E. Gibbs, William G. Poole, Jr., and Paul K. Stockmeyer. An algorithm for reducing the bandwidth and profile of a sparse matrix. *SIAM Journal on Numerical Analysis*, 13(2):236–50, 1976. [2](#), [3](#), [4](#)
- [LLX06] Mingen Lin, Zhiyong Lin, and Jinhui Xu. Graph bandwidth of weighted caterpillars. *Theoretical Computer Science*, 363(3):266–277, 2006. [1](#), [3](#)
- [MS14] Liviu Octavian Maftciu-Scai. The bandwidths of a matrix. A survey of algorithms. *Annals of West University of Timisoara - Mathematics and Computer Science*, 52(2):183–223, 2014. [1](#)
- [NVDSCD24] M. Yassine Naghmouchi, Joseph Vovrosh, Wesley Da Silva Coelho, and Alexandre Dauphin. Optimization of quantum systems emulation via a variant of the bandwidth minimization problem. In *10th International Conference on Control, Decision and Information Technologies*, pages 266–271. IEEE, 2024. [3](#)
- [Pap76] Christos H. Papadimitriou. The NP-completeness of the bandwidth minimization problem. *Computing*, 16(3):263–270, 1976. [1](#)
- [SRB08] Ilya Safro, Dorit Ron, and Achi Brandt. Multilevel algorithms for linear ordering problems. *Journal of Experimental Algorithmics*, 13, 2008. Article 4. [3](#), [4](#)
- [Var25] Luis M. B. Varona. MatrixBandwidth.jl: Fast algorithms for matrix bandwidth minimization and recognition. *Journal of Open Source Software*, 10(116), 2025. Article 9136. [1](#)
- [Var26] Luis M. B. Varona. Heuristic bandwidth reduction algorithms for weighted graphs. East Coast Combinatorics Conference, July 2026. Joint work with Alexander Matheson, <https://luismbvarona.com/assets/slides/eccc-2026.pdf>. [1](#), [3](#)